

Once you've found the address, use it in the `myinit()` function, as follows:

```
kp.addr = (kprobe_opcode_t *)0xc071c9a0;
```

Kprobe executes handler functions before and after the target kernel function is called, and we created the `Pre_Handler()` and `Post_Handler()` functions for this purpose. Assign these to their respective pointer members in the Kprobe struct—`pre_handler` and `post_handler`—in `myinit()`, as you can see. Finally, register your Kprobe with the kernel, with `register_kprobe(&kp);`.

Then compile the module by running `make`:

```
# make
make -C /lib/modules/2.6.34/build SUBDIRS=/root/kprobe modules
make[1]: Entering directory '/root/linux-2.6.34'
CC [M] /root/kprobe/mod1.o
Building modules, stage 2.
MODPOST 1 modules
CC /root/kprobe/mod1.mod.o
LD [M] /root/kprobe/mod1.ko
make[1]: Leaving directory '/root/linux-2.6.34'
```

When done, you are ready to test your example module by inserting it into the kernel:

```
# insmod mod1.ko
```

Confirm that the module is successfully inserted:

```
# lsmod | head -n 5
Module              Size  Used by
mod1                 984    0
fuse                 46627  2
sunrpc              158985  1
xt_physdev           1355  1
```

Now, since you have used `ip_rcv()` as your target function, you need to invoke it with a simple ping:

```
# ping localhost
```

Run `dmesg` and find your module's messages:

```
module inserted
Pre_Handler: counter=0
Post_Handler: counter=1
Pre_Handler: counter=2
Post_Handler: counter=3
```

As you see, you can probe a kernel address and do instrumentation without recompiling the kernel, as was

required by the simple `printk`. When you are done with your debugging, don't forget to remove the module:

```
# rmmod mod1
```

In the exit function, `myexit()`, the kprobe is unregistered by calling `unregister_kprobe(&kp);`.

However, Kprobe has limits to what you can do with it. In the above example, you have just printed some messages in the handlers; you cannot access the function's arguments with Kprobe. Let's move on to something better.

Probing with Jprobe

For those who like bonus features, Jprobe is another kind of probing technique, which can be used to access the target function's arguments, and thus display what was passed to the function. The basics are the same as that of Kprobe, but this additional feature makes Jprobe an interesting tool. To get the `Jprobe` structure details, look in the file `'include/linux/kprobes.h'`:

```
struct jprobe {
    struct kprobe kp;
    void *entry; /* probe handling code to jump to */
};
```

As you see, it contains a `struct kprobe` member, plus a pointer to store the address of a handler function to jump to.

A Jprobe example

```
#include<linux/module.h>
#include<linux/version.h>
#include<linux/kernel.h>
#include<linux/init.h>
#include<linux/kprobes.h>
#include<net/ip.h>
```

```
int my_handler (struct sk_buff *skb, struct net_device *dev,
struct packet_type *pt, struct net_device *orig_dev){
```

```
    struct iphdr *my_iph;
    u32 S_ip,D_ip;
    my_iph = ip_hdr(skb);
    S_ip = my_iph->saddr;
    D_ip = my_iph->daddr;
    printk("Source IP: %d\n",NIPQUAD_FMT,NIPQUAD(S_ip));
    jprobe_return();
}
```

```
static struct jprobe my_probe;
```

```
int myinit(void)
{
```